

Incorrect QAOA energies after refactor

<https://github.com/qiskit-community/qiskit-aqua/issues/1246>

ROW 53

Incorrect QAOA energies after refactor #1246



Closed



danlky opened on Sep 10, 2020 · edited by danlky

Edits ...

[Previous](#) version of QAOA gives different energy expectation values than the [new, improved](#) version.

Information

- Qiskit Aqua version: 0.7.5
- Python version: 3.8.2
- Operating system: Linux

What is the current behavior?

The energy expectation values are different for same qaoa parameters on different versions of aqua.

Steps to reproduce the problem

Use the following file: https://github.com/danlky/QTensor/blob/master/qttensor/tests/qiskit_qaoa_energy.py

- install aqua 0.6.5
- run the file
- install aqua 0.7.5
- change the api (commented lines 12, 13, change to cost_operator on line 93)
- run the file. The results are different

What is the expected behavior?

Same energies for same parameters on different package versions

Suggested solutions

Possibly the problem is in [#852](#)

Assignees

No one assigned

Labels

type: bug

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

No branches or pull requests

Notifications

Customize

Subscribe

You're not receiving notifications from this thread.

Participants



jlapeyre on Sep 11, 2020

Contributor ...

Just so I know we are on the same page: When I run your script with the needed changes under qiskit 0.7.5 the variable `result`, instead of `12`, is `5.181865101305741`. Is this what you find as well?



danlky on Sep 12, 2020

Author ...

@jlapeyre Yes, I get the same numbers.

Cryoris added **type: bug** on Sep 19, 2020

Cryoris added this to To do in [Operators](#) on Sep 19, 2020



rsln-s on Dec 3, 2020

...

Just wanted to bump this and tagging some active developers in the hope of getting some movement on this [@woodsp-ibm](#) [@Cryoris](#) [@manoelmarques](#). I'm observing the same issue with operator exponentiation behaving inconsistently. The versions between which I observe the difference align with [@danlky](#). I'm not just getting different energies, I'm getting different amplitudes after the QAOA circuit execution; perhaps this is related to [#852](#). The versions I checked are:

```
{'qiskit-terra': '0.16.1', 'qiskit-aer': '0.7.1', 'qiskit-ignis': '0.5.1', 'qiskit-ibmq-provider': '0.11'}
```

and

```
{'qiskit-terra': '0.12.0', 'qiskit-aer': '0.4.0', 'qiskit-ignis': '0.2.0', 'qiskit-ibmq-provider': '0.4.0'}
```

See the example below. It is easy to see that the difference is **not** just a global phase, as shown by hardcoded results on qiskit version 0.23.1 and 0.15.0

```
#!/usr/bin/env python

import networkx as nx
import numpy as np
from scipy.optimize import minimize

import qiskit
if qiskit.__version__ > '0.15.0':
    from qiskit.aqua.algorithms.minimum_eigen_solvers.qaoa.var_form import QAOAVarForm
else:
    from qiskit.aqua.algorithms.adaptive.qaoa.var_form import QAOAVarForm

from qiskit import Aer, execute
from qiskit.quantum_info import Pauli
from qiskit.aqua.operators import WeightedPauliOperator

def get_operator(B, m):
    num_nodes = B.shape[0]
    pauli_list = []
    shift = 0

    for i in range(num_nodes):
        for j in range(num_nodes):
            if i != j:
                x_p = np.zeros(num_nodes*2, dtype=np.bool)
                z_p = np.zeros(num_nodes*2, dtype=np.bool)
                z_p[2*i] = True
                z_p[2*j] = True
                pauli_list.append([-B[i, j] / (8 * m), Pauli(z_p, x_p)])

                x_p = np.zeros(num_nodes*2, dtype=np.bool)
                z_p = np.zeros(num_nodes*2, dtype=np.bool)
                z_p[2*i+1] = True
                z_p[2*j+1] = True
                pauli_list.append([-B[i, j] / (8 * m), Pauli(z_p, x_p)])

                x_p = np.zeros(num_nodes*2, dtype=np.bool)
                z_p = np.zeros(num_nodes*2, dtype=np.bool)
                z_p[2*i] = True
                z_p[2*j] = True
                z_p[2*i+1] = True
                z_p[2*j+1] = True
                pauli_list.append([-B[i, j] / (8 * m), Pauli(z_p, x_p)])

            shift -= B[i, j] / (8 * m)
        else:
            shift -= B[i, j] / (2 * m)

    return WeightedPauliOperator(paulis=pauli_list), shift

elist = [[0,1],[1,2],[2,3],[2,4],[3,4],[4,5],[5,6],[4,6],[0,6]]
G = nx.Graph()
G.add_edges_from(elist)
nnodes = G.number_of_nodes()
nedges = len(elist)
assert(nedges == G.number_of_edges())
B = nx.modularity_matrix(G, nodelist=range(nnodes))

p = 10
C, offset = get_operator(B, nedges)
if qiskit.__version__ > '0.15.0':
    varform = QAOAVarForm(C.to_opflow(), p)
else:
    varform = QAOAVarForm(C, p)

theta = np.array([ 6.07687916,  9.04578039, 14.32475339, -1.83010038,  7.97646292,
 16.09278832,  3.90810118, 18.35957614, 20.29304497, 13.54441652,
  6.69979829,  0.98178805, -3.82011781, 10.4430878 , -7.31619394,
 14.06109113,  3.35586645, -2.39458044,  4.81429126, -7.40598448])

backend = Aer.get_backend('statevector_simulator')
qc = varform.construct_circuit(theta)
sv = execute(qc, backend).result().get_statevector()
```

```

backend = Aer.get_backend('statevector_simulator')
qc = varform.construct_circuit(theta)
sv = execute(qc, backend).result().get_statevector()
print(sv[:50])

qiskitversion23 = np.array([ 0.00653109-0.02011478j, -0.00411685+0.00287849j, -0.00411685+0.00287849j,
0.00237577-0.00250061j, -0.00411685+0.00287849j, 0.00424024+0.00731735j,
-0.00576418-0.00427525j, -0.00604232-0.00299428j, -0.00411685+0.00287849j,
-0.00576418-0.00427525j, 0.00424024+0.00731735j, -0.00604232-0.00299428j,
0.00237577-0.00250061j, -0.00604232-0.00299428j, -0.00604232-0.00299428j,
-0.00818109+0.02014514j, 0.01736855+0.00452826j, 0.00303006+0.00285723j,
0.00140519+0.00102003j, -0.00291529-0.0014721j, -0.00052092-0.00256031j,
-0.00346051-0.00377572j, 0.00151463+0.00367647j, -0.00777547-0.00451316j,
0.00181391+0.00344044j, 0.01050733-0.00558139j, 0.00108205+0.01084175j,
0.00140662+0.014539j, -0.00510857-0.00163514j, 0.00293244-0.00081495j,
0.00529303-0.0031521j, -0.0002988 +0.0125395j, 0.01736855+0.00452826j,
0.00140519+0.00102003j, 0.00303006+0.00285723j, -0.00291529-0.0014721j,
0.00181391+0.00344044j, 0.00108205+0.01084175j, 0.01050733-0.00558139j,
0.00140662+0.014539j, -0.00052092-0.00256031j, 0.00151463+0.00367647j,
-0.00346051-0.00377572j, -0.00777547-0.00451316j, -0.00510857-0.00163514j,
0.00529303-0.0031521j, 0.00293244-0.00081495j, -0.0002988 +0.0125395j,
0.00082766-0.00262053j, 0.0034025 -0.00249914j])

qiskitversion15 = np.array([-0.00202787-3.63978743e-03j, 0.00291385+9.87498777e-04j,
0.00291385+9.87498777e-04j, 0.00361592-1.89979395e-04j,
0.00291385+9.87498777e-04j, 0.0209035 -2.27933868e-02j,
0.00112849-1.77807361e-03j, 0.00255573-1.74179305e-03j,
0.00291385+9.87498777e-04j, 0.00112849-1.77807361e-03j,
0.0209035 -2.27933868e-02j, 0.00255573-1.74179305e-03j,
0.00361592-1.89979395e-04j, 0.00255573-1.74179305e-03j,
0.00255573-1.74179305e-03j, 0.02452327-2.92184778e-02j,
0.00213879+7.07876414e-04j, -0.00257498-1.82554886e-03j,
0.00170252+1.93401283e-03j, 0.00193054+2.25857748e-03j,
0.00444372-1.00737152e-02j, 0.03096655-3.44293741e-02j,
0.00434694-1.64091683e-02j, 0.01394855-1.29804830e-02j,
0.0013423 +9.10147922e-05j, 0.00500238+1.09407198e-03j,
0.00713009-1.38165278e-02j, 0.0065575 +1.52277962e-03j,
0.0005172 -1.21264009e-04j, 0.00172281+2.65230190e-03j,
-0.00164662-4.62580168e-03j, 0.01786296-2.04384904e-02j,
0.00213879+7.07876414e-04j, 0.00170252+1.93401283e-03j,
-0.00257498-1.82554886e-03j, 0.00193054+2.25857748e-03j,
0.0013423 +9.10147922e-05j, 0.00713009-1.38165278e-02j,
0.00500238+1.09407198e-03j, 0.0065575 +1.52277962e-03j,
0.00444372-1.00737152e-02j, 0.00434694-1.64091683e-02j,
0.03096655-3.44293741e-02j, 0.01394855-1.29804830e-02j,
0.0005172 -1.21264009e-04j, -0.00164662-4.62580168e-03j,
0.00172281+2.65230190e-03j, 0.01786296-2.04384904e-02j,
0.00170925-1.10413454e-03j, 0.00317307+1.51899973e-03j])

print(qiskitversion23 / qiskitversion15)

```



danlkv on Dec 3, 2020

Author ...

It looks like the way to get the correct expectation is following:

```

from qiskit.optimization.applications.ising import max_cut
qubitOp, offset = max_cut.get_operator(adjacency_matrix)

E_correct = -(E + offset)

```



rslh-s on Dec 3, 2020

...

@danlkv This does not address the issue with the *amplitudes* being different; try running my example above with `qiskit==0.15.0` and `qiskit==0.23.1`



rslin-s on Dec 3, 2020

...

@dankv This does not address the issue with the *amplitudes* being different; try running my example above with `qiskit==0.15.0` and `qiskit==0.23.1`



dankv on Dec 3, 2020

Author ...

I just wanted to share for anyone that could face this problem. The cost landscape is still similar, so maybe amplitudes are just rotated by some phase



rslin-s on Dec 3, 2020

...

Unless there's a mistake in the code I shared above, this is not just a global phase issue...



dankv on Dec 3, 2020

Author ...

Ah, I see, you already looked at `(qiskitversion23 / qiskitversion15)`. I don't know then:

  rslin-s mentioned this on Dec 11, 2020

 [install from repos automatically \(PEP-508\)](#) rslin-s/Multistart-Methods-for-Quantum-Approximate-Optimization#2



jlapeyre on Jan 28, 2021

Contributor ...

Not 100% sure, but it seems very likely that this issue persists in the code migrated to terra.



rslin-s on Jan 28, 2021

...

I believe @dankv has figured this out: the order of the parameters has been changed.

A workaround is simply to swap beta and gamma:

https://github.com/dankv/QTensor/blob/f172751122bd5930e9dfc13790f89210e483e0b8/qtensor/tests/qiskit_qaoa_energy.py#L98

 1



woodsp-ibm on Jan 28, 2021

Member ...

@rslin-s Thanks for responding and noting the 'fix'. The parameter ordering, of the parameterized circuits, that are now used has indeed been problematic and other issues have arisen from this - one noted here [Qiskit/qiskit#5721](#) which itself links to others that are discussing potential change etc in order to have a better defined ordering behavior.

I think we can consider this closed then?



rslin-s on Jan 28, 2021

...

Yes, it's not a bug per se, just an inconsistency between versions. Perfectly fine to close.

 2

 **rsln-s** mentioned this on Dec 11, 2020

 [install from repos automatically \(PEP-508\)](#) **rsln-s/Multistart-Methods-for-Quantum-Approximate-Optimization#2**



jlapeyre on Jan 28, 2021

Contributor ...

Not 100% sure, but it seems very likely that this issue persists in the code migrated to terra.



rsln-s on Jan 28, 2021

...

I believe **@danlkv** has figured this out: the order of the parameters has been changed.

A workaround is simply to swap beta and gamma:

https://github.com/danlkv/QTensor/blob/f172751122bd5930e9dfc13790f89210e483e0b8/qtensor/tests/qiskit_qaoa_energy.py#L98



woodsp-ibm on Jan 28, 2021

Member ...

@rsln-s Thanks for responding and noting the 'fix'. The parameter ordering, of the parameterized circuits, that are now used has indeed been problematic and other issues have arisen from this - one noted here [Qiskit/qiskit#5721](#) which itself links to others that are discussing potential change etc in order to have a better defined ordering behavior.

I think we can consider this closed then?



rsln-s on Jan 28, 2021

...

Yes, it's not a bug per se, just an inconsistency between versions. Perfectly fine to close.



woodsp-ibm on Jan 28, 2021

Member ...

The reported the output change resulted from a change in parameter ordering across versions to which the code sample was sensitive. Changing the ordering 'fixes' the output. As there is other work ongoing, as linked above, around parameter ordering to improve things, such that this sort of problematic behavior is avoided going forwards, I am closing this as resolved.

  **woodsp-ibm** closed this as **completed** on Jan 28, 2021

  **woodsp-ibm** moved this from To do to Done in **Operators** on Jan 28, 2021